📈 Time Series Forecasting & Interactive Business Intelligence Dashboard

---

📌 Project Overview

This project presents an end-to-end **Time Series Forecasting and Business Intelligence Solution** developed to predict future sales using multiple statistical, machine learning, and deep learning models.

The project compares the forecasting performance of traditional approaches such as **ARIMA** and **Prophet** with advanced machine learning models including **XGBoost**, **LSTM**, and two ensemble approaches (**Weighted Ensemble** and **Dynamic Ensemble**).

To enable business users to explore forecasting insights, an interactive **Power BI Dashboard** was designed with dynamic visualizations, model comparison, forecast analysis, and performance evaluation.

The complete pipeline includes:

- Data preprocessing
- Exploratory Data Analysis (EDA)
- Feature Engineering
- Model Development
- Forecast Generation
- Model Evaluation
- Interactive Dashboard Development
- Performance Comparison
- Business Reporting

🎯 Project Objectives

The primary objectives of this project are to:

- Forecast future sales accurately.
- Compare multiple forecasting algorithms.
- Improve prediction accuracy using ensemble learning.
- Visualize business insights through an interactive dashboard.
- Create a reproducible machine learning pipeline.
- Deliver an industry-standard forecasting solution.

🛠️ Technology Stack

Category	Tools
Programming Language	Python
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Plotly
Statistical Model	ARIMA
Forecasting Model	Prophet
Machine Learning	XGBoost
Deep Learning	TensorFlow, Keras (LSTM)
Model Evaluation	Scikit-learn
Dashboard	Power BI
Development Environment	Jupyter Notebook

📂 Repository Structure

```
Time-Series-Forecasting/
│
├── data/
│   ├── raw/
│   ├── processed/
│   └── external/
│
├── notebooks/
│   ├── 01_EDA.ipynb
│   ├── 02_Preprocessing.ipynb
│   ├── 03_ARIMA.ipynb
│   ├── 04_Prophet.ipynb
│   ├── 05_LSTM.ipynb
│   ├── 06_XGBoost.ipynb
│   ├── 07_Weighted_Ensemble.ipynb
│   ├── 08_Dynamic_Ensemble.ipynb
│   └── 09_Model_Evaluation.ipynb
│
├── models/
│   ├── arima.pkl
```

```
│   ├── prophet.pkl
│   ├── xgboost.pkl
│   ├── lstm_model.h5
│   ├── lstm_architecture.json
│   ├── weighted_ensemble.pkl
│   └── dynamic_ensemble.pkl
│
├── dashboards/
│   ├── PowerBI.pbix
│   └── screenshots/
│
├── reports/
│   ├── Final_Report.pdf
│   └── Project_Report.docx
│
├── requirements.txt
├── README.md
└── .gitignore
```

📊 Dataset Description

The dataset contains historical sales records used for forecasting future demand.

Example Variables

Feature	Description
Date	Transaction Date
Sales	Daily Sales
Month	Month Number
Quarter	Quarter of Year
Lag_1	Previous Day Sales
Lag_7	Previous Week Sales
RollingMean7	7-Day Rolling Average
RollingStd7	7-Day Rolling Standard Deviation

🔍 Exploratory Data Analysis (EDA)

The dataset was analyzed to understand historical sales behavior before model development.

Performed analyses include:

- Missing value analysis
- Duplicate record detection
- Trend visualization
- Seasonal pattern analysis
- Distribution analysis
- Correlation analysis
- Time series decomposition
- Outlier detection

⚙️ Feature Engineering

Several time-based and statistical features were created to improve forecasting performance.

Engineered Features

- Year
- Month
- Week
- Day
- Day of Week
- Quarter
- Lag Features
- Rolling Mean
- Rolling Standard Deviation
- Moving Average
- Trend Indicators

🤖 Models Implemented

1. ARIMA

Traditional statistical forecasting model suitable for stationary time series.

Advantages

- Simple

- Interpretable
- Effective for linear trends

---

2. Prophet

Facebook Prophet automatically models trend, seasonality, and holiday effects.

Advantages

- Handles seasonality well
- Robust to missing values
- Easy to tune

---

3. LSTM

Long Short-Term Memory (LSTM) neural network captures long-term dependencies in sequential data.

Advantages

- Learns temporal relationships
- Handles nonlinear patterns
- Strong performance on complex time series

---

4. XGBoost

Gradient boosting model trained using engineered time-series features.

Advantages

- High prediction accuracy
- Handles nonlinear relationships
- Fast training

---

5. Weighted Ensemble

Combines predictions from multiple forecasting models using predefined weights.

Advantages

- Reduces individual model errors
- Improves stability

- Better generalization

---

6. Dynamic Ensemble

Assigns adaptive weights based on model performance across different forecasting periods.

Advantages

- Adaptive
- Robust
- Improved forecasting accuracy

📈 Model Evaluation Metrics

The following metrics were used to compare forecasting performance:

- RMSE (Root Mean Squared Error)
- MAE (Mean Absolute Error)
- MAPE (Mean Absolute Percentage Error)
- R^2 Score (Coefficient of Determination)

📊 Dashboard Features

The Power BI dashboard provides interactive analytics including:

Dashboard 1

- Executive Overview
- KPI Cards
- Historical Sales Trend
- Monthly Analysis
- Quarterly Analysis
- Forecast Summary

Dashboard 2

- Feature Importance
- Seasonal Analysis
- Model Performance
- Residual Analysis
- Forecast Distribution

Dashboard 3

- Actual vs Predicted Comparison
- Forecast Comparison
- Error Distribution
- Model Ranking
- Dynamic Model Selector
- Performance Metrics

📈 Model Comparison

Model	RMSE	MAE	MAPE	R^2 Score
ARIMA	2.7255	2.1975	6.4543	
Prophet	3.7066	3.1765	--------	
LSTM	0.7575	0.6079	2.2593	
XGBoost	0.1469	0.0926		0.9989
ENSEMBLE	4.4498	3.9217	11.5917	-3.7263

▶️ Installation

Clone the repository:

```
git clone https://github.com/your-username/Time-Series-Forecasting.git
```

Navigate to the project folder:

```
cd Time-Series-Forecasting
```

Install the required dependencies:

```
pip install -r requirements.txt
```

🚀 How to Reproduce the Project

1. Clone the repository.
2. Install the required Python packages.
3. Place the dataset in the `data/raw/` directory.
4. Run the preprocessing notebook.
5. Train the forecasting models.
6. Generate forecasts and evaluation metrics.
7. Save the trained models in the `models/` folder.
8. Open the Power BI dashboard (`.pbix`) to explore the results.

📁 Saved Models

The repository includes trained models for reuse:

- `arima.pkl`
- `prophet.pkl`
- `xgboost.pkl`
- `lstm_model.h5`
- `lstm_architecture.json`
- `weighted_ensemble.pkl`
- `dynamic_ensemble.pkl`

📚 Future Enhancements

Potential improvements include:

- Hyperparameter optimization
- Transformer-based forecasting models
- Real-time prediction API
- Automated model retraining
- Cloud deployment
- Interactive web dashboard
- CI/CD integration
- Docker containerization

🤝 Acknowledgements

Special thanks to the mentors, instructors, and the open-source community for providing the libraries and tools that made this project possible.

👩‍💻 Author

Madhuri Kumari

Machine Learning | Data Science | Time Series Forecasting | Business Intelligence

GitHub: https://github.com/Madhuri-Kumari2504

LinkedIn: https://www.linkedin.com/in/madhuri-kumari-20071332b/

⭐ If you found this project useful, consider giving it a star on GitHub!